

K Means Clustering

September 22, 2026

=====

Algorithm and Demo

K-Means Clustering

Dr. Osita Onyejekwe

University of Colorado Boulder

=====

Outline

What is clustering?

K-Means Clustering:

K-Means Clustering: Demo

K-Means Clustering: The Algorithm

Picking K

Shortcomings of K Means

Clustering Methods

Often times we deal with observations for which there is no target or response variable. So, there is no way to do any predictions (at least not directly), as we don't have a target variable to predict. In these situations, however, what we instead try to do is uncover some structure within the data, and this is where clustering methods enter the picture.

Clustering methods fall into the realm of **unsupervised machine-learning techniques** and refer to a set of techniques designed to identify subgroups or clusters within a data set. In other words, the idea is to partition a data set into subgroups in such a way that the members within each group are as similar as possible to one another and quite distinct from the members of other groups.

A natural question is what exactly do we mean by similar? The answer depends on the data set we are dealing with as well as the type of method we use. In what follows we will discuss one of the simplest and widely used clustering method, namely K-Means Clustering.

K-Means Clustering: Demo

```
[1]: import pandas as pd
```

```
[2]: import numpy as np
```

```
[3]: import seaborn as sns
import matplotlib.pyplot as plt
```

```
[4]: from sklearn.datasets import make_blobs # artifical data set
```

First, let's create some artificial blobs!

```
[5]: # only 2 features but in real life there is WAY more features (dimensional
    ↪reduciton techniques)

def create_dataframe(feat_names, n_feat, n_samp, centers, std, random_state =1):
    X,y = make_blobs(n_samples = n_samp, centers = centers, cluster_std=std,
    ↪n_features= n_feat,
                        random_state = random_state, center_box = (0,6))
    cancer = pd.DataFrame()
    for name, i in zip(feat_names, range(n_feat)):
        cancer[name] = X[:,i]
    return cancer, y
```

```
[6]: # code here

df, y = create_dataframe(feat_names=['X1', 'X2'], n_feat=2, n_samp=500, std=0.
    ↪3, centers=4, random_state=30)
```

Suppose we are given the following unlabeled (no response variable) data set.

```
[7]: # code here
df.head()
```

```
[7]:
```

	X1	X2
0	4.216186	1.703706
1	5.407564	1.727611
2	6.199649	2.300265
3	3.803320	2.270811
4	5.377644	2.022812

Since this is a basic example with only two features, let's plot a scatterplot and see what this data looks like. Remember that in real life we almost always deal with higher dimensional data sets so visualizing them may not be possible (at least not without some more work!).

```
[8]: # looks like there are 3-4 clusters ( i know that there are 4)

plt.figure(figsize=(12,8))
plt.scatter(df['X1'], df['X2'])
```

```
plt.xlabel('X1\n\n Figure 1', fontsize=14)
plt.ylabel('X2', fontsize=14)
plt.show()
```

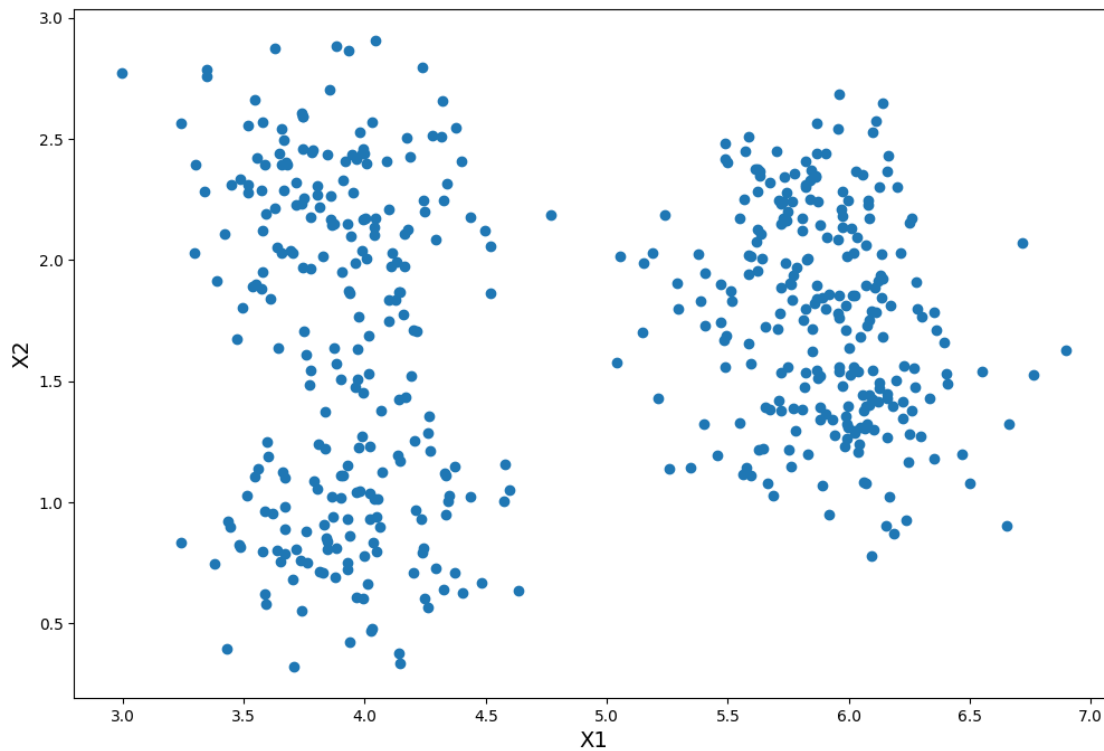


Figure 1

From Figure 1 above, it is apparent that there are three different subgroups/clusters in our data. K-Means clustering is a simple way to help us identify these 3-4. In other words, it will be able to tell us which observations fall into which cluster. It also enables us to properly assign any new observations that we may obtain in the future.

```
[9]: from matplotlib.animation import FuncAnimation
     from matplotlib import animation
     from matplotlib import animation, rc
     from IPython.display import HTML
```

```
[10]: from sklearn.cluster import KMeans
```

```
[ ]: '''
     plt.figure(figsize=(22,84))

     m=11
     for i in range(1,m):
```

```

    km=KMeans(n_clusters=4, max_iter=2*i-1,init='random', n_init=1,
↳random_state=101)
    km.fit(df)
    labels=km.labels_
    centers=km.cluster_centers_

    plt.subplot(m,2,i+1)

    plt.scatter(df['X1'],df['X2'],c=labels, cmap='coolwarm')
    plt.scatter(x=centers[:,0],y=centers[:,1],color='r', marker='v',s=82)
    plt.title("# of Iterations {}".format(2*i-1), fontsize=18)

plt.subplot(m,2,1)
plt.scatter(df['X1'],df['X2'],c=y, cmap='coolwarm')#
    plt.title("Original",fontsize=18)

plt.subplot(m,2,m+1)
plt.scatter(df['X1'],df['X2'],c=y, cmap='coolwarm')
plt.title("Original Data",fontsize=18)

# plt.show()

'''

```

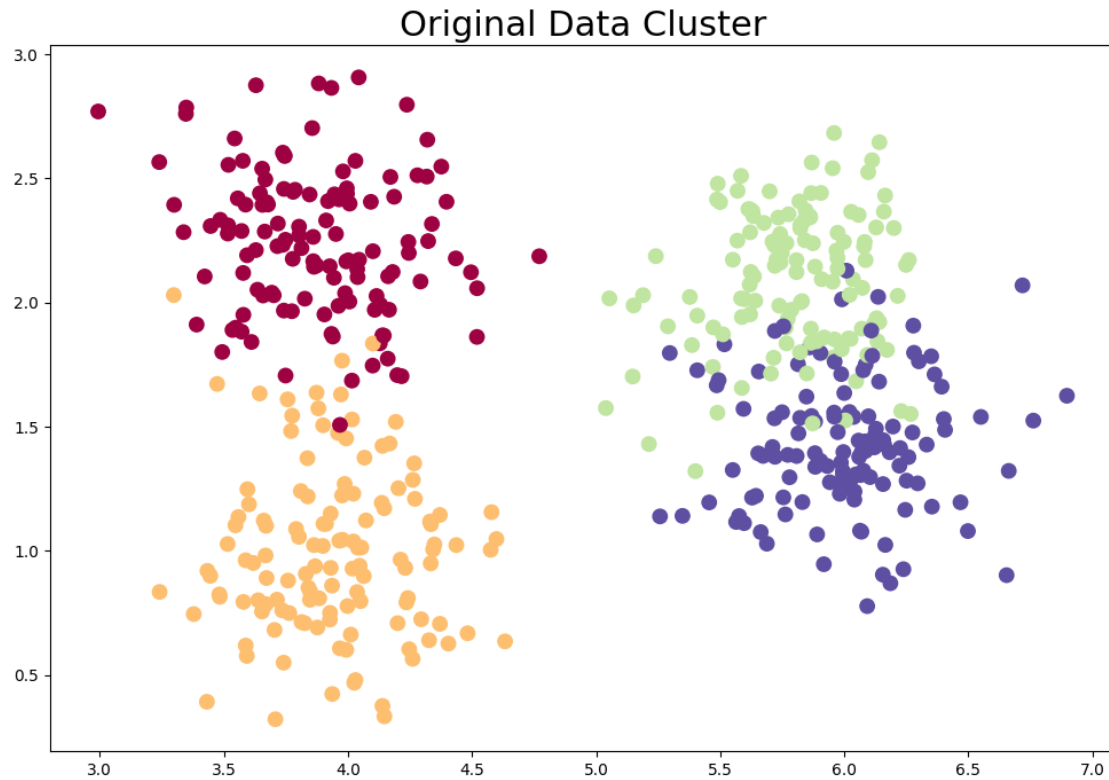
```

[11]: # we observe here same data data but note that theyh are colored according to
↳the appropdiat cluster, but be aware that I would definety not
# know this in real life

fig = plt.figure(figsize = (12,8))

plt.scatter(df['X1'], df['X2'], c=y, s=80, cmap = 'Spectral')
plt.title("Original Data Cluster", fontsize = 22)
plt.show()

```



```
[12]: # This code runs k-means clusters where it first runs one iteration and then it
      ↪ runs 2,3,4 all the way to 10 iterations

      # note that this scatter plot, despite the coloring is exactly the same as the
      ↪ previous scatter plot but once we
      # run the simulation you will see in real time how k-means clustering
      ↪ re-assigns the colors in order to come up with
      # the picture above.

      %matplotlib ipynpl
      fig = plt.figure(figsize=(12,8))

      #def init():
      #    scat=plt.scatter(df['X1'],df['X2'], c=np.random.randn(0,4,df.shape[0]))
      #    return scat,

      def run_knn(m):

          def update_plot(i):

              if i==0:
```

```

        labels=np.random.randint(0,4,df.shape[0])
        scat=plt.scatter(df['X1'],df['X2'],c=labels,cmap='Spectral')

    else:
        km=KMeans(n_clusters=4, max_iter=i,init='random', n_init=1,
↳random_state=1)
        km.fit(df)
        labels=km.labels_
        inertia=km.inertia_
        centers=km.cluster_centers_
        if (i==1):
            cent=plt.scatter(x=centers[:,0],y=centers[:,1],color='r',
↳marker='^',s=200)
            elif i==m-1:
                cent=plt.scatter(x=centers[:,0],y=centers[:,1],color='r',
↳marker='x',lw=5,s=200)
            else:
                cent=plt.scatter(x=centers[:,0],y=centers[:,1],color='b',
↳marker='v',s=100)

        scat=plt.scatter(df['X1'],df['X2'],c=labels,cmap='Spectral')
        plt.title("Iteration {} \n Inertia {:.3f}".
↳format(i,inertia),fontsize=22)

    return scat,

def main():

    fig = plt.figure(figsize=(12,8))

    scat = plt.scatter(df['X1'],df['X2'],s=80)

    anim = FuncAnimation(fig, update_plot,frames=m, interval=1000,
↳blit=True)

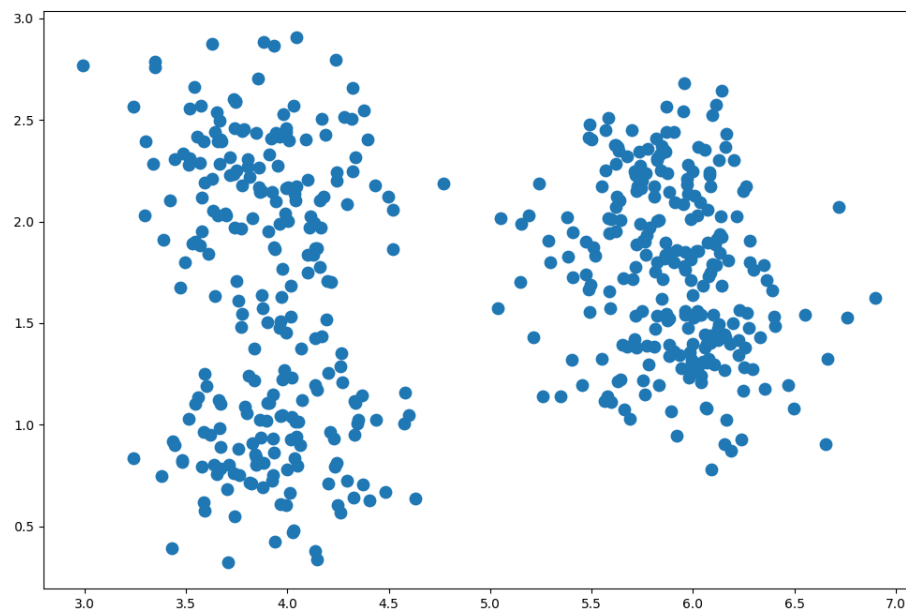
    return anim

return main()

anim = run_knn(11)
#run_knn(11)
# anim.save('knn.gif', writer='PillowWriter')

```

```
#HTML(anim.to_html5_video())
```



How does K-Means clustering work?

The K-Means clustering method is quite intuitive and simple. Let $\{I\}_i^K$ denote the collection of sets containing the indices of each of the observations in our data. We require that these sets satisfy the following conditions

$$\bigcup_i I_i = \{1, \dots, n\}, \text{ and } I_i \cap I_j = \emptyset \forall i \neq j$$

We are going to consider a good clustering if the variation within each cluster is as small as it can possibly be.

If we denote the within-cluster variation for cluster I_k by $V(I_k)$ then we would want to solve the following optimization problem

$$\min_{I_k, k=1, \dots, K} \left\{ \sum_{k=1}^K V(I_k) \right\}$$

In other words, we would want to find the right index sets which result in the lowest total within-cluster variation.

There are many ways to quantify the within-cluster variation, but perhaps the simplest one is to just take the Euclidean distance. In other words,

$$V(I_k) = \frac{1}{|I_k|} \sum_{i, j \in I_k} \|\vec{x}^{(i)} - \vec{x}^{(j)}\|^2$$

where

$$\|\vec{x}^{(i)} - \vec{x}^{(j)}\|^2 = \sum_{k=1}^p (x_k^{(i)} - x_k^{(j)})^2$$

In layman's terms, we want to group together observations that have the smallest Euclidean distance from one another.

The Algorithm

Finding a global solution to the optimization problem above is a difficult problem, since there are essentially K^n ways to partition n observations into K clusters. Searching over this solution space is not feasible especially if K and n are large. The K-Means Algorithm finds a local optimum to the optimization problem above, which for many practical purposes proves to be good enough.

Below is a description of the algorithm:

Step 1.

Randomly initialize cluster assignments. That is, to each observation assign randomly a number between 1 and K .

Step 2.

Iterate the following steps until the cluster assignments no longer change:

A. For each of the K clusters compute the cluster centroid; that is the means of the observations assigned to each cluster. For example, the centroid for cluster I_j would be

$$\bar{x}^{(j)} = \frac{1}{|I_j|} \sum_{\bar{x}_i \in I_j} \bar{x}_i$$

B. Assign each observation to the cluster whose Euclidean distance to its centroid is smallest.

How to pick K ?

One coarse method of picking K is via the elbow trick. Basically we plot the inertia score as a function of K and pick the K that lies at the elbow of the graph.

Let's illustrate it with the example above

```
[13]: # code here
```

```
inertia_score = []
for i in range(2,15):
    km = KMeans(n_clusters = i)
    km.fit(df)
    inertia_score.append(km.inertia_)
```

```
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
```

```

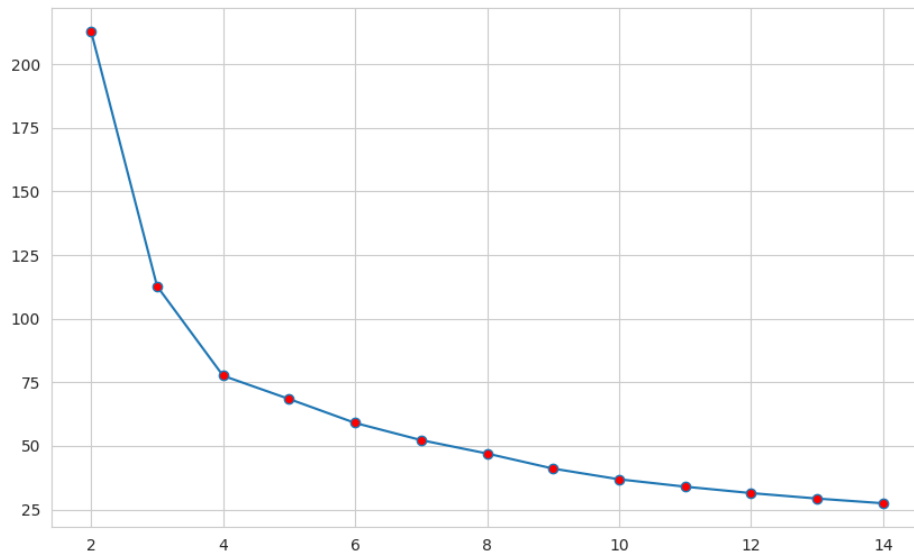
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
/opt/conda/lib/python3.10/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(

```

```

[14]: # codehere
plt.figure(figsize = (10,6))
sns.set_style('whitegrid')
plt.plot(range(2,15), inertia_score, marker = 'o', markerfacecolor='r')
plt.show()

```



We observe that until $K = 4$ there is a significant drop in the inertia score, but after that it levels off. So, in this case, even if we didn't know to begin with that our data set had four distinct clusters in it, it would have been reasonable to pick $K = 4$. Sometimes, this is not as easy to spot and other more sophisticated ways are required to pick the right K .

Another such method is the silhouette score.

Limitations of K-Means Method

Despite the fact that $K - Means$ is a fast and very useful method to identify subgroups within data, it has its shortcomings.

- A. You must specify the number of clusters before you run the algorithm. Obviously this is a challenge, since in unsupervised learning we don't know apriori how many or if we have any clusters in our data to begin with.
- B. It performs poorly when clusters are of varying sizes, shapes, or densities. For example, if one cluster is of spherical shape, and others have a more elongated ellipsoidal shape, K-Means will most likely perform poorly as it simply measures distances between points and is unable to pick up on such trends.
- C. It performs poorly on observations that lie on the boundary between two clusters.